

Facial Recognition in Images:
Facial Segmentation, Feature Extraction, and Classification

Noah Hester and Christian Mason and Samson Seligman

COMP/MATH/PHYS 445: Image Analysis and Pattern Recognition

Dr. William Barber

May 17, 2025

Abstract

The goal of our project was to create two classifiers that would be used in a larger facial recognition system. After gathering several hundred images of faces and standardizing them to be used in our models, we applied two different techniques for classification, first by the PCA method and second by a neural network. The PCA method calculates basis vectors for a feature space and projects images into that space for comparison and classification using 1-nearest neighbor. We created our neural network using Tensorflow with keras, with our final model consisting of five layers. The results were mostly positive, showing the majority of our images were matched appropriately, although there were several misclassifications in both systems that could likely be the result of sparsity in the data. We concluded that both classifiers are advantageous to a facial recognition system, and both offer qualities that are desirable in differing circumstances.

Introduction

The goal of our project was to create two classifiers that would be used in a larger facial recognition system. While last class we focused on segmenting images for features, this class project is focused on taking those segmented features and making decisions based on their discrepancies. After gathering several hundred images of faces and standardizing them to be used in our models, we applied two different techniques for classification, first by using a principle component analysis (PCA) and second by a neural network. Images were organized and given class labels for the PCA method (a supervised learning model), and then labels were one-hot vector encoded for processing within the neural network (another supervised model). Both models were designed to take the testing images and make as accurate an assessment as it could about the class of that image based on a moderate amount of training data. A small output GUI

was created for direct visualization of the attempted matches in each case. The main areas of focus for this project will be image processing and dataset creation, classification by principal component analysis and k-nearest neighbor, and classification by neural network.

Methods

Operating Systems:

- MacOS Sequoia (Version 15.1.1)
- Windows 11 (Version 10.0.22631)

Hardware Specifications:

- 2023 Macbook Air
 - Processor: Apple M2
 - RAM: 8 GB
- 2022 HP Pavilion Gaming Laptop
 - Processor: Intel Core i5-10300H
 - GPU: NVIDIA GTX
 - RAM: 8 GB

Software Tools and Versions:

- Code Editor:
 - PyCharm 2024.3.1.1 (Professional Edition)
 - Visual Studio Code (Version 1.95.3)
- Programming Languages:
 - Python (Version 3.12)

Dependencies and Libraries:

- Numpy (Version 2.2.6)
- OpenCV (Version 4.11.0.86)
- Tensorflow (Version 2.19.0)
- Pillow (Version 11.2.1)

Before any classification or feature extraction can be applied, the images must be processed and made uniform for the dataset in order for standardization among inputs. The original images may have multiple faces within them and so, for processing, we first want to cut out windows with only the face since this is the specific data we want to process and extract features from. To do this, we utilized the Viola-Jones method with Haar classifiers to determine face windows. These windows were then formatted as grayscale and placed into folders for classification. This makes reference classes easy since the classes can be assigned from the folder names upon data retrieval. Each of these windows was also reshaped to 128x128 so that all images, regardless of resolution, would have the same shape. In this way we can process all of them the same, and get samples of varying resolution for more accurate classification in extraneous conditions. These images were then read into the program and finally flattened into a 1x16384 vector before going into any of the classifying systems. About 100 images were taken and processed for each subject to be recognized and even more faces were processed into the training set to establish a more broad “unknown” class for if the face is not close enough to any of the ones we wish to recognize.

The first method that was applied to the image data was principal component analysis. This was done in order to create a projection matrix for the feature space of the images. Once projected into this space, we can compare based on the features that contain the most variance

among the image set. The process for creating this projection matrix followed the method described in “Facial Feature Extraction Techniques for Face Recognition” (Abiyev, 2). By this method, in order to create this projection matrix, the mean of all the images was computed and a matrix was formed with the variance of the images (differences from the mean) as the columns. This was then used to calculate the covariance matrix by multiplying the matrix by its transpose like: $C = A \times A^T$. The eigenvectors are then computed from this resulting covariance matrix and used as the principal components. However, this computation ultimately amounts to multiplying matrices and getting a result of a 16384 x 16384 matrix. This becomes very computationally intensive as we must solve the eigenvalue problem: $CU = U\Lambda$ with this massive matrix, where Λ is a diagonal matrix with the eigenvalues, and U has the associated eigenvectors. So, in order to calculate these without dealing in giant matrices, we followed an alternative method by performing the reverse multiplication: $M = A^T \times A$. The eigenvectors were then extracted from this much smaller matrix (whose width and height are equal the size of the training set). These eigenvectors were then transformed into our desired vectors by multiplication by the original set of variances (previously referenced as A). The top 50 eigenvectors were then taken in order to ideally capture the basis vectors for the features with the most variance in the images. In order to actually extract features in this space for comparison between images, we multiply the centered images in the training and testing sets (centered meaning having subtracted the mean from the image) by the projection matrix formed from the eigenvectors. This projects the images into the feature space defined by the basis eigenvectors. The test images are then compared to all the training images using a euclidean distance within the feature space (amounting to something similar to the Mahalanobis distance within regular image space), and an association is made with the closest training image being the best match

according to the system. The classification is done by essentially a 1-nearest neighbor algorithm where the closest training image to the test image in the feature space determines the class of the test image. In this way we can associate a match image and take on the same class in order to recognize test faces similar to those in the training set.

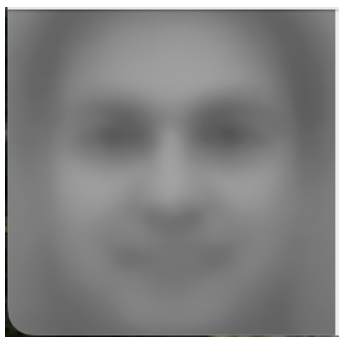
For our neural network we used tensorflow with keras. After a few attempts with different amounts of epochs and layers with different activation functions, we were able to alter some of the hyperparameters in order to return a consistent strong result for the output layer. The final model (for now) has five layers consisting of an input layer, two dense layers (one with a reLU activation function and one with a tanh activation function), followed by a dropout layer and finally the output layer (using a softmax activation). Images were originally fed in after the PCA method had transformed them into a “fifty feature” eigenspace version of the original, but this was difficult to achieve and unnecessary. We began feeding the images in from the point of the viola jones preprocessing and normalization/flattening and this had the intended effects and was adopted as the final method. The first two dense layers have activation functions which are geared towards feature detection, the dropout layer helps minimize some of the error, and the softmax at the end helps us by outputting a percentage of confidence for each class. We can then take the most likely or confident node to be the output. The system was trained and tested on the same images as our PCA classifier. Training would ultimately consist of 200 epochs, although we found some improvement past 100, and more will be required for the live feed version, alongside more data in order to create full fledged models. A method to save the training data results (in order to preserve the shifted weights and skip the time retraining the model) was created and can be further updated to augment and append those results (weights) rather than retrain on large data sets in full. A live version of this system was then created, utilizing the

neural network for fast face class prediction, and though it was successful, it is quite sensitive and not entirely accurate much of the time. An increased amount of training data with different angles could likely improve results.

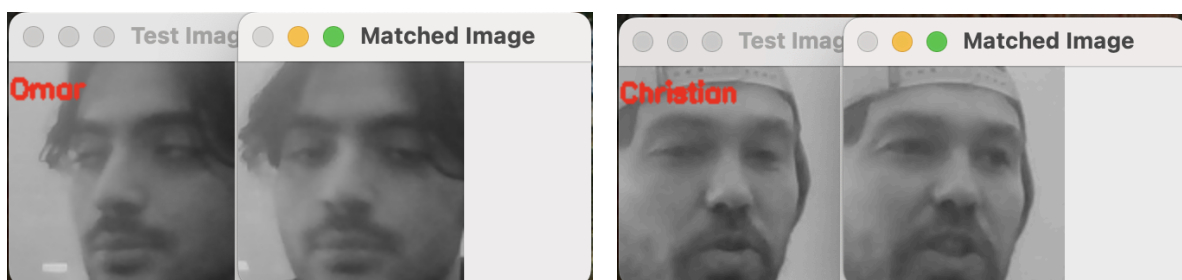
Results

The results were quite promising, being mostly positive, showing the majority of our images were matched appropriately, although there were several misclassifications in both systems that could likely be the result of sparsity in the data. The PCA method was able to make some classifications accurately that our neural network classifier was unable to accurately assess. The neural network functioned well after proper activation functions were given and a correct amount of epochs were undertaken. Prior to this it was unable to identify any class with reliability. Afterwards, it was able to correctly classify some images that the PCA method missed, but also had some misclassifications of its own.

The PCA method requires calculating the mean of all the images in the dataset. This produced an interesting vague face-like image where we can see the average features of hundreds of faces:

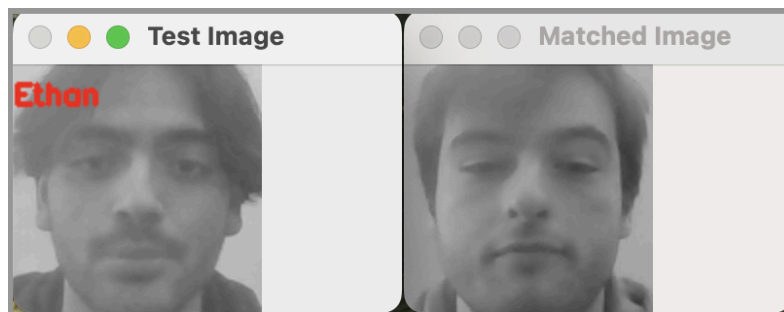
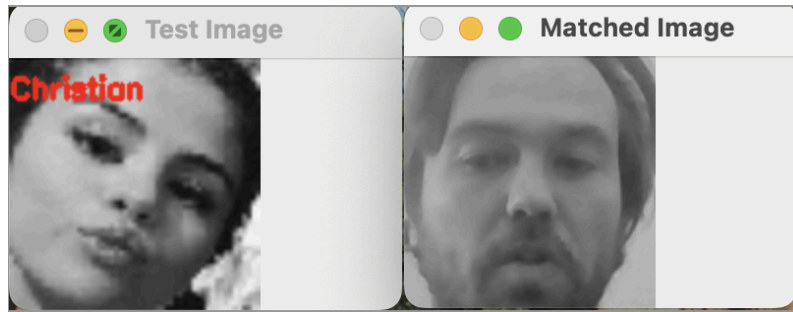


Below are some examples of the system correctly guessing the class of the test images:



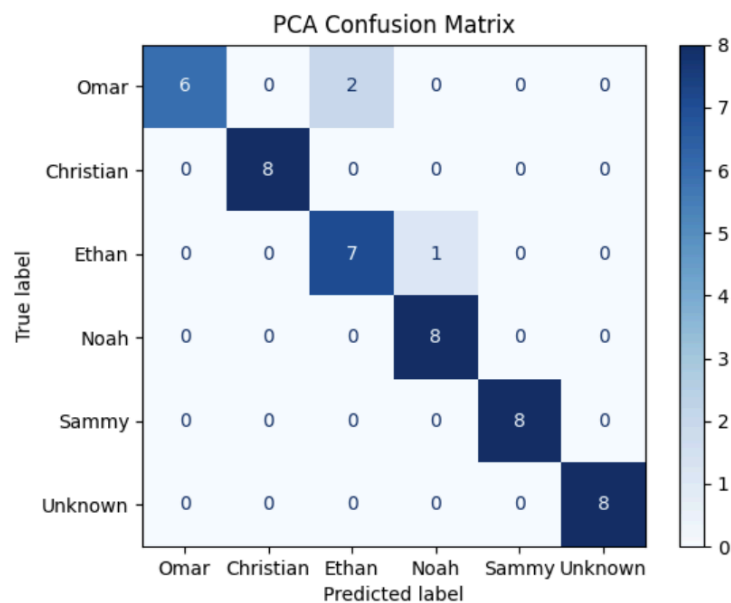
Here we can see the training image associated with the input test image and the assigned class shown in red on the test image window.

Below are some misclassifications where the system assigned the wrong class to an image:



In the second set we can see the similarity in the gray values between the images, and so we can see how the system might mistake one for being of the same class as the other.

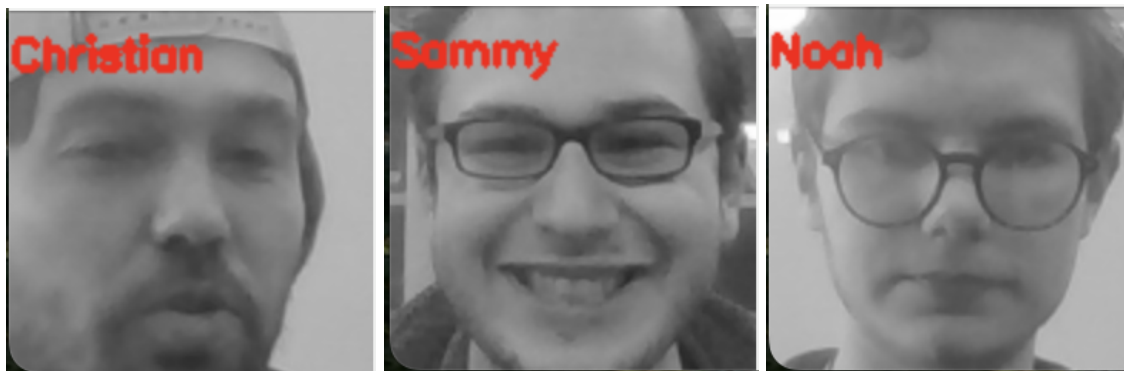
Below is the resulting confusion matrix after testing with 8 test images from each class:



We can see from this matrix that it identified most images correctly, missing two of Omar's, and one of Ethan's.

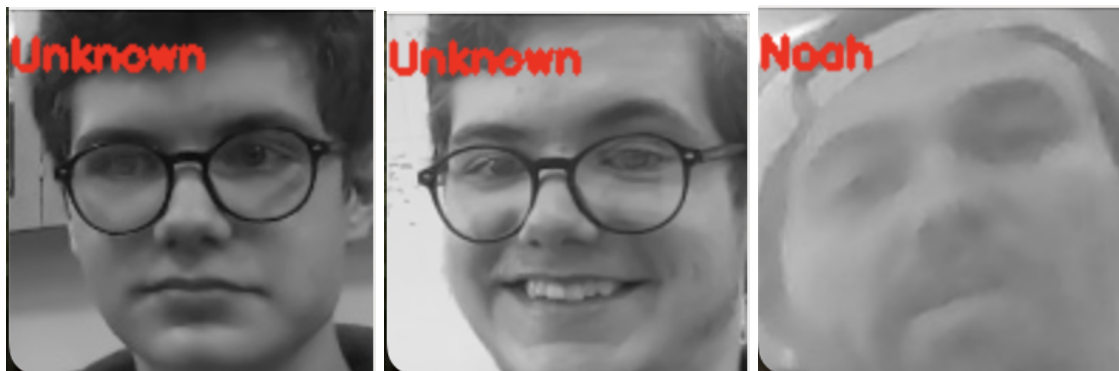
The neural network required much less calculation before being passed into the system, but the neural network did need to be trained on the training set before prediction. It was ultimately trained for about 200 epochs on the training set and then its weights were saved as a *.weights.h5 file so that the network would not need to be retrained.

Below are some examples of the neural network correctly classifying images:



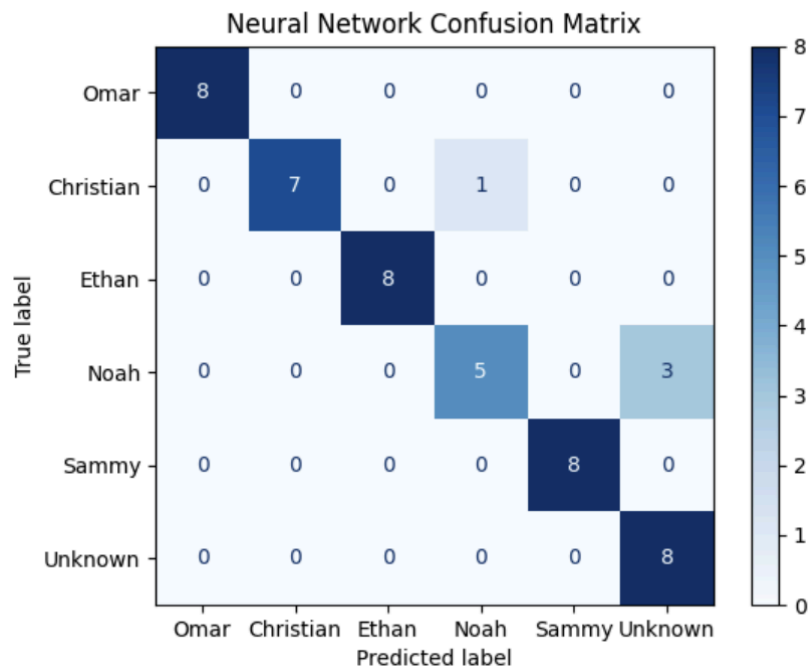
Here the system does not perform image association, and only outputs a percentage vector. So this interface simply displays the test image with the predicted label placed on it in red.

Below are some sample outputs of misclassification in the system:



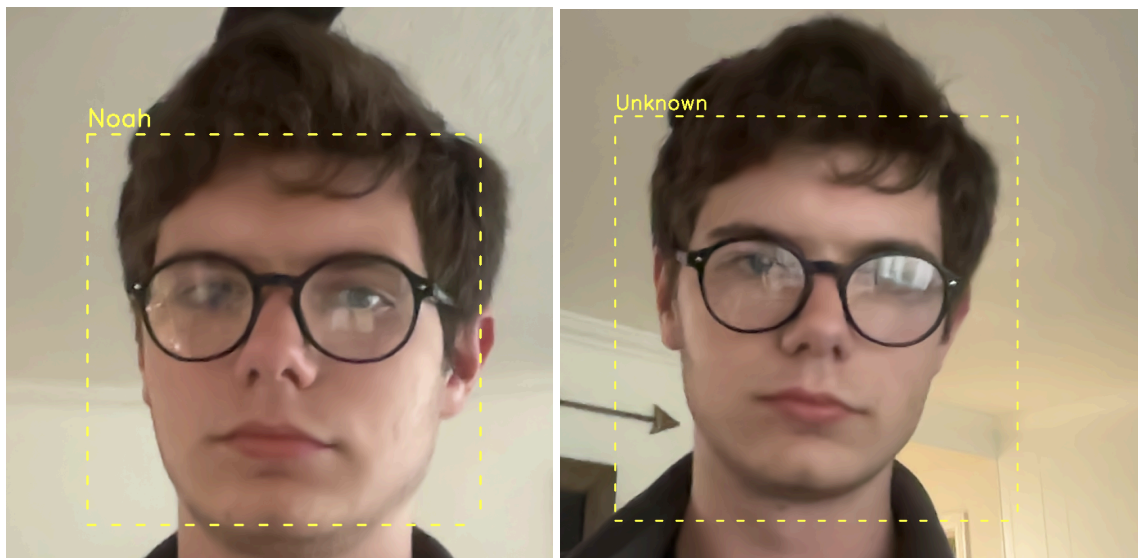
We can see it quite often was unable to identify Noah and misclassified Christian as Noah.

Below is the confusion matrix for the neural network classifier based on the same 8 test images:



Here we can see that the network classified the majority of images correctly, but missed three of Noah and one of Christian.

The live system was successful at times, but not very stable or accurate. Below are examples of correct and incorrect classifications on a live camera feed.



The results were flickering between classes, but more data might help in improving live results.

Conclusion

Both classifiers are advantageous to a facial recognition system, and both offer qualities that are desirable in differing circumstances, while it appears the neural method may offer some ability to work in an upscaled system slightly faster since it doesn't have to actively check against the hundreds of images in the training set, it may be the case the PCA method offers a degree of accuracy that it cannot offer, at least when dealing with smaller amounts of data. In theory, one might be able to achieve better accuracy by combining the systems in some way so that they complement each other and might be able to make up for some of each other's errors. This might increase the processing time of the system, especially with large data sets in the PCA, but might also lend itself towards improving the accuracy of the system by quite a bit. Another method of improvement for perhaps the live version might be sampling and taking multiple classifications over time to average the best classification or tracking and comparing to faces in previous frames. Ultimately, the greatest improvement would likely arise from simply larger scale mass data collection in each person class and in the unknown faces class. These all might contribute to better classification in the future.

References

The full code and images may be found in the GitHub repository here:

<https://github.com/noahcoder11/COMP-445-Final-Project>

Abiyev, R. H. (2014, December 23). Facial feature extraction techniques for face recognition.

Journal of Computer Science.

<https://thescipub.com/abstract/10.3844/jcssp.2014.2360.2365>